Dokumentasi Resmi & Sintaks C++ (Modern C++20/C++23 Up-to-

Bahasa: C++ | Total Chunk/Dokumen: 74 | Disiapkan untuk RAG System

[cpp_001] C++ - Modern C++ Overview

Welcome Back to C++ - Modern C++ (C++20/23 Idioms, RAII, auto): Overview

Sumber: ht

Welcome back to C++ - Modern C++

Since its creation, C++ has become one of the most widely used programming languages in the world. Well-written C++ programs are fast and efficient. The language is more flexible than other languages: It can work at the highest levels of abstraction, and down at the level of the silicon.

C++ supplies highly optimized standard libraries. It enables access to low-level hardware features, to maximize speed and minimize memory requirements. C++ can create almost any kind of program: Games, device drivers, High-Performance Computing, cloud, desktop, embedded, and mobile apps, and much more. Even libraries and compilers for other programming languages are written in C++.

One of the original requirements for C++ was backward compatibility with the C language. As a result, C++ permits C-style programming, with raw pointers, arrays, null-terminated character strings, and other features. They can enable great performance, but can also spawn bugs and complexity.

The evolution of C++ emphasizes features that greatly reduce the need to use C-style idioms. The old C-programming facilities are still there when you need them. However, in modern C++ code you should need them less and less. Modern C++ code is simpler, safer, more elegant, and still as fast as ever.

The following sections provide an overview of the main features of modern C++. Unless noted otherwise, the features listed here are available in C++11 and later. In the Microsoft C++ compiler, you can set the `/std` compiler option to specify which version of the standard to use for your project.

[cpp_002] C++ - Modern C++ Overview

Welcome Back to C++ - Modern C++ (C++20/23 Idioms, RAII, auto): Resources and smart pointers

Sumber: ht

Resources and smart pointers

One of the major classes of bugs in C-style programming is the *memory leak*. Leaks are often caused by a failure to call `delete` for memory that was allocated with `new`. Modern C++ emphasizes the principle of *resource acquisition is initialization* (RAII).

The idea is simple. Resources, such as heap memory, file handles, and sockets, should be *owned* by an object. That object creates, or receives, the newly allocated resource in its constructor, and deletes it in its destructor. The principle of RAII guarantees that all resources get properly returned to the operating system when the owning object goes out of scope.

To support easy adoption of RAII principles, the C++ Standard Library provides three smart pointer types:

- `std::unique_ptr`
- `std::shared_ptr`
- `std::weak_ptr`

A *smart pointer* handles the allocation and deletion of the memory it owns. The following example shows a class with an array member that is allocated on the heap in the call to `make_unique()`. The `unique_ptr` class encapsulates the calls to `new` and `delete`. When a `widget` object goes out of scope, the `unique_ptr` destructor will be invoked and it releases the memory that was allocated for the array.

Whenever possible, use a smart pointer to manage heap memory. If you must use the `new` and `delete` operators explicitly, follow the principle of RAII. For more information, see Object lifetime and resource management (RAII).

Contoh Kode / Perintah CLI:

```
#include <memory>
```

```
class widget
{
private:
    std::unique_ptr<int[]> data;
public:
    widget(const int size) { data = std::make_unique<int[]>(size); }
    void do_something() {}
};

void functionUsingWidget() {
    widget w(1000000);  // lifetime automatically tied to enclosing scope
                        // constructs w, including the w.data gadget member
    // ...
    w.do_something();
    // ...
} // automatic destruction and deallocation for w and w.data
```

[cpp_003] C++ - Modern C++ Overview

Welcome Back to C++ - Modern C++ (C++20/23 Idioms, RAII, auto): `std::string` and `std::string_view`

Sumber: ht

`std::string` and `std::string_view`
C-style strings are another major source of bugs. By using `std::string` and `std::wstring`, you can eliminate virtually all the errors associated with C-style strings. You also gain the benefit of member functions for searching, appending, prepending, and so on. Both are highly optimized for speed. When passing a string to a function that requires only read-only access, in C++17 you can use `std::string_view` for even greater performance benefit.

[cpp_004] C++ - Modern C++ Overview

Welcome Back to C++ - Modern C++ (C++20/23 Idioms, RAII, auto): `std::vector` and other Standard Library containers

Sumber: ht

`std::vector` and other Standard Library containers
The standard library containers all follow the principle of RAII. They provide iterators for safe traversal of elements. They're highly optimized for performance and thoroughly tested for correctness. By using these containers, you eliminate the potential for bugs or inefficiencies that might be introduced in custom data structures. Instead of raw arrays, use `vector` as a sequential container in C++.
Use `map` (not `unordered_map`) as the default associative container. Use `set`, `multimap`, and `multiset` for degenerate and multi cases.
When you need performance optimization, consider using:
- Unordered associative containers such as `unordered_map`. These containers have lower per-element overhead and constant-time lookup, but they can be harder to use correctly and efficiently.
- Sorted `vector`. For more information, see Algorithms.
Don't use C-style arrays. For older APIs that need direct access to the data, use accessor methods such as `f(vec.data(), vec.size());` instead. For more information about containers, see C++ Standard Library Containers.

Contoh Kode / Perintah CLI:

```
vector<string> apples;
apples.push_back("Granny Smith");

// ----------------------------------------

map<string, string> apple_color;
// ...
apple_color["Granny Smith"] = "Green";
```

[cpp_005] C++ - Modern C++ Overview

Welcome Back to C++ - Modern C++ (C++20/23 Idioms, RAII, auto): Standard Library algorithms

Sumber: ht

Standard Library algorithms

Before you assume that you need to write a custom algorithm for your program, review the C++ Standard Library algorithms. The Standard Library contains an ever-growing assortment of algorithms for many common operations such as searching, sorting, filtering, and randomizing. The math library is extensive. In C++17 and later, parallel versions of many algorithms are provided.

Here are some important examples:

- `for_each`: the default traversal algorithm, along with range-based `for` loops.
- `transform`: for not-in-place modification of container elements.
- `find_if`: the default search algorithm.
- `sort`, `lower_bound`, and the other default sorting and searching algorithms.

To write a comparator, use strict `<` and use *named lambdas* when you can.

Contoh Kode / Perintah CLI:

```
auto comp = [](const widget& w1, const widget& w2)
     { return w1.weight() < w2.weight(); }

sort( v.begin(), v.end(), comp );

auto i = lower_bound( v.begin(), v.end(), widget{0}, comp );
```

[cpp_006] C++ - Modern C++ Overview

Welcome Back to C++ - Modern C++ (C++20/23 Idioms, RAII, auto): `auto` instead of explicit type names

Sumber: ht

`auto` instead of explicit type names

C++11 introduced the `auto` keyword for use in variable, function, and template declarations. `auto` tells the compiler to deduce the type of the object so that you don't have to type it explicitly. `auto` is especially useful when the deduced type is a nested template:

Contoh Kode / Perintah CLI:

```
map<int,list<string>>::iterator i = m.begin(); // C-style
auto i = m.begin(); // modern C++
```

[cpp_007] C++ - Modern C++ Overview

Welcome Back to C++ - Modern C++ (C++20/23 Idioms, RAII, auto): Range-based `for` loops

Sumber: ht

Range-based `for` loops

C-style iteration over arrays and containers is prone to indexing errors and is also tedious to type. To eliminate these errors, and make your code more readable, use range-based `for` loops with both Standard Library containers and raw arrays. For more information, see Range-based `for` statement.

Contoh Kode / Perintah CLI:

```
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> v {1,2,3};
```

```
    // C-style
    for(int i = 0; i < v.size(); ++i)
    {
        std::cout << v[i];
    }

    // Modern C++:
    for(auto& num : v)
    {
        std::cout << num;
    }
}
```

[cpp_008] C++ - Modern C++ Overview

Welcome Back to C++ - Modern C++ (C++20/23 Idioms, RAII, auto): `constexpr` expressions instead of macros

Sumber: ht

`constexpr` expressions instead of macros

Macros in C and C++ are tokens that are processed by the preprocessor before compilation. Each instance of a macro token is replaced with its defined value or expression before the file is compiled. Macros are commonly used in C-style programming to define compile-time constant values. However, macros are error-prone and difficult to debug. In modern C++, you should prefer `constexpr` variables for compile-time constants:

Contoh Kode / Perintah CLI:

```
#define SIZE 10 // C-style
constexpr int size = 10; // modern C++
```

[cpp_009] C++ - Modern C++ Overview

Welcome Back to C++ - Modern C++ (C++20/23 Idioms, RAII, auto): Uniform initialization

Sumber: ht

Uniform initialization

In modern C++, you can use brace initialization for any type. This form of initialization is especially convenient when initializing arrays, vectors, or other containers. In the following example, `v2` is initialized with three instances of `S`. `v3` is initialized with three instances of `S` that are themselves initialized using braces. The compiler infers the type of each element based on the declared type of `v3`.

For more information, see Brace initialization.

Contoh Kode / Perintah CLI:

```
#include <vector>

struct S
{
    std::string name;
    float num;
    S(std::string s, float f) : name(s), num(f) {}
};

int main()
{
    // C-style initialization
    std::vector<S> v;
    S s1("Norah", 2.7);
    S s2("Frank", 3.5);
    S s3("Jeri", 85.9);

    v.push_back(s1);
    v.push_back(s2);
```

```
    v.push_back(s3);

    // Modern C++:
    std::vector<S> v2 {s1, s2, s3};

    // or...
    std::vector<S> v3{ {"Norah", 2.7}, {"Frank", 3.5}, {"Jeri", 85.9} };

}
```

[cpp_010] C++ - Modern C++ Overview

Welcome Back to C++ - Modern C++ (C++20/23 Idioms, RAII, auto): Move semantics

Sumber: ht

Move semantics
Modern C++ provides *move semantics*, which make it possible to eliminate unnecessary memory copies. In earlier versions of the language, copies were unavoidable in certain situations. A *move* operation transfers ownership of a resource from one object to the next without making a copy. Some classes own resources such as heap memory, file handles, and so on.
When you implement a resource-owning class, you can define a *move constructor* and *move assignment operator* for it. The compiler chooses these special members during overload resolution in situations where a copy isn't needed. The Standard Library container types invoke the move constructor on objects if one is defined. For more information, see Move Constructors and Move Assignment Operators (C++).

[cpp_011] C++ - Modern C++ Overview

Welcome Back to C++ - Modern C++ (C++20/23 Idioms, RAII, auto): Lambda expressions

Sumber: ht

Lambda expressions
In C-style programming, a function can be passed to another function by using a *function pointer*. Function pointers are inconvenient to maintain and understand. The function they refer to might be defined elsewhere in the source code, far away from the point at which it's invoked. Also, they're not type-safe.
Modern C++ provides *function objects*, classes that override the `operator()` operator, which enables them to be called like a function. The most convenient way to create function objects is with inline lambda expressions. The following example shows how to use a lambda expression to pass a function object that the `find_if` function invokes on each element in the vector:
The lambda expression `= { return i > x && i < y; }` can be read as "function that takes a single argument of type `int` and returns a boolean that indicates whether the argument is greater than `x` and less than `y`." Notice that the variables `x` and `y` from the surrounding context can be used in the lambda. The `[=]` specifies that those variables are *captured* by value. In other words, the lambda expression has its own copies of those values.

Contoh Kode / Perintah CLI:

```
std::vector<int> v {1,2,3,4,5};
    int x = 2;
    int y = 4;
    auto result = find_if(begin(v), end(v), [=](int i) { return i > x && i < y; });
```

[cpp_012] C++ - Modern C++ Overview

Welcome Back to C++ - Modern C++ (C++20/23 Idioms, RAII, auto): Exceptions

Sumber: ht

Exceptions
Modern C++ emphasizes exceptions, not error codes, as the best way to report and handle error conditions. For more information, see Modern C++ best practices for exceptions and error handling.

[cpp_013] C++ - Modern C++ Overview

Welcome Back to C++ - Modern C++ (C++20/23 Idioms, RAII, auto): `std::atomic`

Sumber: ht

`std::atomic`
Use the C++ Standard Library `std::atomic` struct and related types for inter-thread communication mechanisms.

[cpp_014] C++ - Modern C++ Overview

Welcome Back to C++ - Modern C++ (C++20/23 Idioms, RAII, auto): `std::variant` (C++17)

Sumber: ht

`std::variant` (C++17)
Unions are commonly used in C-style programming to conserve memory by enabling members of different types to occupy the same memory location. Unions aren't type-safe and are prone to programming errors. C++17 introduces the `std::variant` class as a more robust and safe alternative to unions. The `std::visit` function can be used to access the members of a `variant` type in a type-safe manner.

[cpp_015] C++ - Modern C++ Overview

Welcome Back to C++ - Modern C++ (C++20/23 Idioms, RAII, auto): See also

Sumber: ht

See also
- C++ Language Reference
- Lambda Expressions
- C++ Standard Library
- Microsoft C/C++ language conformance

[cpp_016] C++ - Modern Type System

C++ Type System (Fundamental types, const, auto, decltype, Value categories): Overview

Sumber: ht

C++ type system
The concept of *type* is important in C++. Every variable, function argument, and function return value must have a type in order to be compiled. Also, all expressions (including literal values) are implicitly given a type by the compiler before they're evaluated. Some examples of types include built-in types such as **`int`** to store integer values, **`double`** to store floating-point values, or Standard Library types such as class `std::basic_string` to store text. You can create your own type by defining a **`class`** or **`struct`**. The type specifies the amount of memory that's allocated for the variable (or expression result). The type also specifies the kinds of values that may be stored, how the compiler interprets the bit patterns in those values, and the operations you can perform on them. This article contains an informal overview of the major features of the C++ type system.

[cpp_017] C++ - Modern Type System

C++ Type System (Fundamental types, const, auto, decltype, Value categories): Terminology

Sumber: ht

Terminology
Scalar type: A type that holds a single value of a defined range. Scalars include arithmetic types (integral or floating-point values), enumeration type members, pointer types, pointer-to-member types, and `std::nullptr_t`. Fundamental types are typically scalar types.
Compound type: A type that isn't a scalar type. Compound types include array types, function types, class (or struct) types, union types, enumerations, references, and pointers to non-static class members.
Variable: The symbolic name of a quantity of data. The name can be used to access the data it refers to throughout the scope of the code where it's defined. In C++, *variable* is often used to refer to instances of scalar data types, whereas instances of other types are typically called *objects*.
Object: For simplicity and consistency, this article uses the term *object* to refer to any instance of a class or structure. When it's used in the general sense, it includes all types, even scalar variables.
POD type (plain old data): This informal category of data types in C++ refers to types that are scalar (see the Fundamental types section) or are *POD classes*. A POD class has no static data members that aren't also PODs, and has no user-defined constructors, user-defined destructors, or user-defined assignment operators. Also, a POD class has no virtual functions, no base class, and no private or protected non-static data members. POD types are often used for external data interchange, for example with a module written in the C language (which has POD types only).

[cpp_018] C++ - Modern Type System

C++ Type System (Fundamental types, const, auto, decltype, Value categories): Specifying variable and function types

Sumber: ht

Specifying variable and function types
C++ is both a *strongly typed* language and a *statically typed* language; every object has a type and that type never changes. When you declare a variable in your code, you must either specify its type explicitly, or use the **`auto`** keyword to instruct the compiler to deduce the type from the initializer. When you declare a function in your code, you must specify the type of its return value and of each argument. Use the return value type **`void`** if no value is returned by the function. The exception is when you're using function templates, which allow for arguments of arbitrary types.
After you first declare a variable, you can't change its type at some later point. However, you can copy the variable's value or a function's return value into another variable of a different type. Such operations are called *type conversions*, which are sometimes necessary but are also potential sources of data loss or incorrectness.
When you declare a variable of POD type, we strongly recommend you *initialize* it, which means to give it an initial value. Until you initialize a variable, it has a "garbage" value that consists of whatever bits happened to be in that memory location previously. It's an important aspect of C++ to remember, especially if you're coming from another language that handles initialization for you. When you declare a variable of non-POD class type, the constructor handles initialization.
The following example shows some simple variable declarations with some descriptions for each. The example also shows how the compiler uses type information to allow or disallow certain subsequent operations on the variable.

Contoh Kode / Perintah CLI:

```
int result = 0;              // Declare and initialize an integer.
double coefficient = 10.8;   // Declare and initialize a floating
                             // point value.
auto name = "Lady G.";       // Declare a variable and let compiler
                             // deduce the type.
auto address;                // error. Compiler cannot deduce a type
                             // without an intializing value.
age = 12;                    // error. Variable declaration must
                             // specify a type or use auto!
```

```
result = "Kenny G.";          // error. Can't assign text to an int.
string result = "zero";       // error. Can't redefine a variable with
                              // new type.
int maxValue;                 // Not recommended! maxValue contains
                              // garbage bits until it is initialized.
```

[cpp_019] C++ - Modern Type System

C++ Type System (Fundamental types, const, auto, decltype, Value categories): Fundamental (built-in) types

Sumber: ht

Fundamental (built-in) types

Unlike some languages, C++ has no universal base type from which all other types are derived. The language includes many *fundamental types*, also known as *built-in types*. These types include numeric types such as **`int`**, **`double`**, **`long`**, **`bool`**, plus the **`char`** and **`wchar_t`** types for ASCII and UNICODE characters, respectively. Most integral fundamental types (except **`bool`**, **`double`**, **`wchar_t`**, and related types) all have **`unsigned`** versions, which modify the range of values that the variable can store. For example, an **`int`**, which stores a 32-bit signed integer, can represent a value from -2,147,483,648 to 2,147,483,647. An **`unsigned int`**, which is also stored as 32 bits, can store a value from 0 to 4,294,967,295. The total number of possible values in each case is the same; only the range is different.

The compiler recognizes these built-in types, and it has built-in rules that govern what operations you can perform on them, and how they can be converted to other fundamental types. For a complete list of built-in types and their size and numeric limits, see Built-in types.

The following illustration shows the relative sizes of the built-in types in the Microsoft C++ implementation:

:::image type="content" source="../cpp/media/built-intypesizes.png" alt-text="Diagram showing the relative sizes in bytes of C++ built-in types: bool and char (1 byte), short (2 bytes), int, long, and float (4 bytes), double and long long (8 bytes)":::

The following table lists the most frequently used fundamental types, and their sizes in the Microsoft C++ implementation:

Type	Size	Comment
`int`	4 bytes	The default choice for integral values.
`double`	8 bytes	The def

[cpp_020] C++ - Modern Type System

C++ Type System (Fundamental types, const, auto, decltype, Value categories): The `void` type

Sumber: ht

The `void` type

The **`void`** type is a special type; you can't declare a variable of type **`void`**, but you can declare a variable of type `void *` (pointer to **`void`**), which is sometimes necessary when allocating raw (untyped) memory. However, pointers to **`void`** aren't type-safe and their use is discouraged in modern C++. In a function declaration, a **`void`** return value means that the function doesn't return a value; using it as a return type is a common and acceptable use of **`void`**. While the C language required functions that have zero parameters to declare **`void`** in the parameter list, for example, `fn(void)`, this practice is discouraged in modern C++; a parameterless function should be declared `fn()`. For more information, see Type conversions and type safety.

[cpp_021] C++ - Modern Type System

C++ Type System (Fundamental types, const, auto, decltype, Value categories): `const` type qualifier

Sumber: ht

`const` type qualifier

Any built-in or user-defined type may be qualified by the **`const`** keyword. Additionally, member functions may be **`const`**-qualified and even **`const`**-overloaded. The value of a **`const`** type can't be modified after it's initialized.

The **`const`** qualifier is used extensively in function and variable declarations and "const correctness" is an important concept in C++; essentially it means to use **`const`** to guarantee, at compile time, that values aren't modified unintentionally. For more information, see `const`.

A **`const`** type is distinct from its non-**`const`** version; for example, **`const int`** is a distinct type from **`int`**. You can use the C++ **`const_cast`** operator on those rare occasions when you must remove *const-ness* from a variable. For more information, see Type conversions and type safety.

Contoh Kode / Perintah CLI:

```
const double PI = 3.1415;
PI = .75; //Error. Cannot modify const variable.
```

[cpp_022] C++ - Modern Type System

C++ Type System (Fundamental types, const, auto, decltype, Value categories): String types

Sumber: ht

String types

Strictly speaking, the C++ language has no built-in string type; **`char`** and **`wchar_t`** store single characters - you must declare an array of these types to approximate a string, adding a terminating null value (for example, ASCII `'\0'`) to the array element one past the last valid character (also called a *C-style string*). C-style strings required much more code to be written or the use of external string utility library functions. But in modern C++, we have the Standard Library types `std::string` (for 8-bit **`char`**-type character strings) or `std::wstring` (for 16-bit **`wchar_t`**-type character strings). These C++ Standard Library containers can be thought of as native string types because they're part of the standard libraries that are included in any conformant C++ build environment. Use the `#include ` directive to make these types available in your program. (If you're using MFC or ATL, the `CString` class is also available, but isn't part of the C++ standard.) The use of null-terminated character arrays (the C-style strings previously mentioned) is discouraged in modern C++.

[cpp_023] C++ - Modern Type System

C++ Type System (Fundamental types, const, auto, decltype, Value categories): User-defined types

Sumber: ht

User-defined types

When you define a **`class`**, **`struct`**, **`union`**, or **`enum`**, that construct is used in the rest of your code as if it were a fundamental type. It has a known size in memory, and certain rules about how it can be used apply to it for compile-time checking and, at runtime, for the life of your program. The primary differences between the fundamental built-in types and user-defined types are as follows:

- The compiler has no built-in knowledge of a user-defined type. It learns of the type when it first encounters the definition during the compilation process.
- You specify what operations can be performed on your type, and how it can be converted to other types, by defining (through overloading) the appropriate operators, either as class members or non-member functions. For more information, see Function overloading

[cpp_024] C++ - Modern Type System

C++ Type System (Fundamental types, const, auto, decltype, Value categories): Pointer types

Sumber: ht

Pointer types

As in the earliest versions of the C language, C++ continues to let you declare a variable of a pointer type by using the special declarator **`*`** (asterisk). A pointer type stores the address of the location in memory where the actual data value is stored. In modern C++, these pointer types are referred to as *raw pointers*, and they're accessed in your code through special operators: **`*`** (asterisk) or **`->`** (dash with greater-than, often called *arrow*). This memory access operation is called *dereferencing*. Which operator you use depends on whether you're dereferencing a pointer to a scalar, or a pointer to a member in an object.

Working with pointer types has long been one of the most challenging and confusing aspects of C and C++ program development. This section outlines some facts and practices to help use raw pointers if you want to. However, in modern C++, it's no longer required (or recommended) to use raw pointers for object ownership at all, due to the evolution of the smart pointer (discussed more at the end of this section). It's still useful and safe to use raw pointers for observing objects. However, if you must use them for object ownership, you should do so with caution and with careful consideration of how the objects owned by them are created and destroyed.

The first thing that you should know is that a raw pointer variable declaration only allocates enough memory to store an address: the memory location that the pointer refers to when it's dereferenced. The pointer declaration doesn't allocate the memory needed to store the data value. (That memory is also called the *backing store*.) In other words, by declaring a raw pointer variable, you're creating a memory address variable, not an actual data variable. If you derefere

Contoh Kode / Perintah CLI:

```
int* pNumber;       // Declare a pointer-to-int variable.
*pNumber = 10;      // error. Although this may compile, it is
                    // a serious error. We are dereferencing an
                    // uninitialized pointer variable with no
                    // allocated memory to point to.

// ----------------------------------------

int number = 10;          // Declare and initialize a local integer
                              // variable for data backing store.
    int* pNumber = &number;   // Declare and initialize a local integer
                              // pointer variable to a valid memory
                              // address to that backing store.
...
    *pNumber = 41;            // Dereference and store a new value in
                              // the memory pointed to by
                              // pNumber, the integer variable called
                              // "number". Note "number" was changed, not
                              // "pNumber".

// ----------------------------------------

void someFunction() {
    unique_ptr<MyClass> pMc(new MyClass);
    pMc->DoSomeWork();
}
  // No memory leak. Out-of-scope automatically calls the destructor
  // for the unique_ptr, freeing the resource.
```

[cpp_025] C++ - Modern Type System

C++ Type System (Fundamental types, const, auto, decltype, Value categories): Windows data types

Sumber: ht

Windows data types
In classic Win32 programming for C and C++, most functions use Windows-specific typedefs and `#define` macros (defined in `windef.h`) to specify the types of parameters and return values. These Windows data types are mostly special names (aliases) given to C/C++ built-in types. For a complete list of these typedefs and preprocessor definitions, see Windows Data Types. Some of these typedefs, such as `HRESULT` and `LCID`, are useful and descriptive. Others, such as `INT`, have no special meaning and are just aliases for fundamental C++ types. Other Windows data types have names that are retained from the days of C programming and 16-bit processors, and have no purpose or meaning on modern hardware or operating systems. There are also special data types associated with the Windows Runtime Library, listed as Windows Runtime base data types. In modern C++, the general guideline is to prefer the C++ fundamental types unless the Windows type communicates some extra meaning about how the value is to be interpreted.

[cpp_026] C++ - Modern Type System

C++ Type System (Fundamental types, const, auto, decltype, Value categories): More information

Sumber: ht

More information
For more information about the C++ type system, see the following articles.
Value types\
Describes *value types* along with issues relating to their use.
Type conversions and type safety\
Describes common type conversion issues and shows how to avoid them.

[cpp_027] C++ - Modern Type System

C++ Type System (Fundamental types, const, auto, decltype, Value categories): See also

Sumber: ht

See also
Welcome back to C++\
C++ language reference\
C++ Standard Library

[cpp_028] C++ - Memory Management - Smart Pointers

Smart Pointers (std::unique_ptr, std::shared_ptr, std::weak_ptr, make_unique): Overview

Sumber: ht

Smart pointers (Modern C++)
In modern C++ programming, the Standard Library includes *smart pointers*, which are used to help ensure that programs are free of memory and resource leaks and are exception-safe.

[cpp_029] C++ - Memory Management - Smart Pointers

Smart Pointers (std::unique_ptr, std::shared_ptr, std::weak_ptr, make_unique): Uses for smart pointers

Sumber: ht

Uses for smart pointers

Smart pointers are defined in the `std` namespace in the \ header file. They're crucial to the *Resource Acquisition Is Initialization* (RAII) programming idiom. The main goal of this idiom is to ensure that resource acquisition occurs at the same time that the object is initialized. All resources for the object are created and made ready in one line of code.

In practical terms, the main principle of RAII is to give ownership of any heap-allocated resource to a stack-allocated object whose destructor contains the code to delete or free the resource and also any associated cleanup code. Such objects include dynamically allocated memory or system object handles.

In most cases, when you initialize a raw pointer or resource handle to point to an actual resource, pass the pointer to a smart pointer immediately. In modern C++, raw pointers are only used in small code blocks of limited scope, loops, or helper functions where performance is critical and there's no chance of confusion about ownership.

The following example compares a raw pointer declaration to a smart pointer declaration.

!code-cpp[smart_pointers_intro#1]

As shown in the example, a smart pointer is a class template that you declare on the stack, and initialize by using a raw pointer that points to a heap-allocated object. After the smart pointer is initialized, it owns the raw pointer. This approach means that the smart pointer is responsible for deleting the memory that the raw pointer specifies.

The smart pointer destructor contains the call to `delete`. Because the smart pointer is declared on the stack, its destructor is invoked when the smart pointer goes out of scope. It's invoked even if an exception is thrown somewhere further up the stack.

Access the encapsulated pointer by us

[cpp_030] C++ - Memory Management - Smart Pointers

Smart Pointers (std::unique_ptr, std::shared_ptr, std::weak_ptr, make_unique): Kinds of smart pointers

Sumber: ht

Kinds of smart pointers

The following section summarizes the different kinds of smart pointers that are available in the Windows programming environment and describes when to use them.

[cpp_031] C++ - Memory Management - Smart Pointers

Smart Pointers (std::unique_ptr, std::shared_ptr, std::weak_ptr, make_unique): C++ Standard Library smart pointers

Sumber: ht

C++ Standard Library smart pointers

Use these smart pointers as a first choice for encapsulating pointers to plain old C++ objects (POCO).

- `unique_ptr`

Allows exactly one owner of the underlying pointer. Use as the default choice for POCO unless you know for certain that you require a `shared_ptr`. Can be moved to a new owner, but not copied or shared. Replaces `auto_ptr`, which is deprecated. Compare to `boost::scoped_ptr`.

`unique_ptr` is small and efficient. The size is one pointer and it supports rvalue references for fast insertion and retrieval from C++ Standard Library collections. Header file: ``. For more information, see How to: Create and use unique_ptr instances and unique_ptr Class.

- `shared_ptr`

Reference-counted smart pointer. Use when you want to assign one raw pointer to multiple owners. For example, you can return a copy of a pointer from a container but also keep the original. The raw pointer isn't deleted until all `shared_ptr` owners go out of scope or otherwise give up ownership.

The size is two pointers: one for the object and one for the shared control block that contains the reference count.

Header file: ``. For more information, see How to: Create and use shared_ptr instances and shared_ptr Class.
- `weak_ptr`
Special-case smart pointer for use with `shared_ptr`. A `weak_ptr` provides access to an object that one or more `shared_ptr` instances own, but doesn't participate in reference counting. Use when you want to observe an object, but don't require it to remain alive. Required in some cases to break circular references between `shared_ptr` instances.
Header file: ``. For more information, see How to: Create and use weak_ptr instances and weak_ptr Class.

[cpp_032] C++ - Memory Management - Smart Pointers

Smart Pointers (std::unique_ptr, std::shared_ptr, std::weak_ptr, make_unique): Smart pointers for COM objects (classic Windows programming)

Sumber: ht

Smart pointers for COM objects (classic Windows programming)
When you work with COM objects, wrap the interface pointers in an appropriate smart pointer type. The Active Template Library (ATL) defines several smart pointers for various purposes. You can also use the `_com_ptr_t` smart pointer type, which the compiler uses when it creates wrapper classes from .tlb files. It's the best choice when you don't want to include the ATL header files.
- CComPtr Class
Use this class unless you can't use ATL. Performs reference counting by using the `AddRef` and `Release` methods. For more information, see How to: Create and use CComPtr and CComQIPtr instances.
- CComQIPtr Class
Resembles `CComPtr` but also provides simplified syntax for calling `QueryInterface` on COM objects. For more information, see How to: Create and use CComPtr and CComQIPtr instances.
- CComHeapPtr Class
Smart pointer to objects that use `CoTaskMemFree` to free memory.
- CComGITPtr Class
Smart pointer for interfaces that are obtained from the global interface table (GIT).
- _com_ptr_t Class
Resembles `CComQIPtr` in functionality but doesn't depend on ATL headers.

[cpp_033] C++ - Memory Management - Smart Pointers

Smart Pointers (std::unique_ptr, std::shared_ptr, std::weak_ptr, make_unique): ATL smart pointers for POCO objects

Sumber: ht

ATL smart pointers for POCO objects
In addition to smart pointers for COM objects, ATL also defines smart pointers, and collections of smart pointers, for plain old C++ objects (POCO). In classic Windows programming, these types are useful alternatives to the C++ Standard Library collections, especially when code portability isn't required or when you don't want to mix the programming models of the C++ Standard Library and ATL.
- CAutoPtr Class
Smart pointer that enforces unique ownership by transferring ownership on copy. Comparable to the deprecated `std::auto_ptr` Class.
- CHeapPtr Class
Smart pointer for objects that are allocated by using the C malloc function.
- CAutoVectorPtr Class
Smart pointer for arrays that are allocated by using `new[]`.
- CAutoPtrArray Class
Class that encapsulates an array of `CAutoPtr` elements.
- CAutoPtrList Class
Class that encapsulates methods for manipulating a list of `CAutoPtr` nodes.

[cpp_034] C++ - Memory Management - Smart Pointers

Smart Pointers (std::unique_ptr, std::shared_ptr, std::weak_ptr, make_unique): See also

Sumber: ht

See also
- Pointers
- C++ Language Reference
- C++ Standard Library

[cpp_035] C++ - Exception Handling & RAII

Modern C++ Best Practices for Exceptions and Error Handling: Overview

Sumber: ht

Modern C++ best practices for exceptions and error handling
In modern C++, in most scenarios, the preferred way to report and handle both logic errors and runtime errors is to use exceptions. It's especially true when the stack might contain several function calls between the function that detects the error, and the function that has the context to handle the error. Exceptions provide a formal, well-defined way for code that detects errors to pass the information up the call stack.

[cpp_036] C++ - Exception Handling & RAII

Modern C++ Best Practices for Exceptions and Error Handling: Use exceptions for exceptional code

Sumber: ht

Use exceptions for exceptional code
Program errors are often divided into two categories:
- Logic errors caused by programming mistakes. For example, an "index out of range" error.
- Runtime errors that are beyond the control of programmer. For example, a "network service unavailable" error.
In C-style programming and in COM, error reporting is managed either by returning a value that represents an error code or a status code for a particular function, or by setting a global variable that the caller may optionally retrieve after every function call to see whether errors were reported. For example, COM programming uses the `HRESULT` return value to communicate errors to the caller. And the Win32 API has the `GetLastError` function to retrieve the last error reported by the call stack. In both of these cases, it's up to the caller to recognize the code and respond to it appropriately. If the caller doesn't explicitly handle the error code, the program might crash without warning. Or, it might continue to execute using bad data and produce incorrect results.
Exceptions are preferred in modern C++ for the following reasons:
- An exception forces calling code to recognize an error condition and handle it. Unhandled exceptions stop program execution.
- An exception jumps to the point in the call stack that can handle the error. Intermediate functions can let the exception propagate. They don't have to coordinate with other layers.
- The exception stack-unwinding mechanism destroys all objects in scope after an exception is thrown, according to well-defined rules.
- An exception enables a clean separation between the code that detects the error and the code that handles the error.
The following simplified example shows the necessary syntax for throwing and catching exceptio

Contoh Kode / Perintah CLI:

```
#include <stdexcept>
#include <limits>
#include <iostream>

using namespace std;

void MyFunc(int c)
{
    if (c > numeric_limits< char> ::max())
```

```
    {
        throw invalid_argument("MyFunc argument too large.");
    }
    //...
}

int main()
{
    try
    {
        MyFunc(256); //cause an exception to throw
    }

    catch (invalid_argument& e)
    {
        cerr << e.what() << endl;
        return -1;
    }
    //...
    return 0;
}
```

[cpp_037] C++ - Exception Handling & RAII

Modern C++ Best Practices for Exceptions and Error Handling: Basic guidelines

Sumber: ht

Basic guidelines

Robust error handling is challenging in any programming language. Although exceptions provide several features that support good error handling, they can't do all the work for you. To realize the benefits of the exception mechanism, keep exceptions in mind as you design your code.

- Use asserts to check for conditions that should always be true or always be false. Use exceptions to check for errors that might occur, for example, errors in input validation on parameters of public functions. For more information, see the Exceptions versus assertions section.
- Use exceptions when the code that handles the error is separated from the code that detects the error by one or more intervening function calls. Consider whether to use error codes instead in performance-critical loops, when code that handles the error is tightly coupled to the code that detects it.
- For every function that might throw or propagate an exception, provide one of the three exception guarantees: the strong guarantee, the basic guarantee, or the nothrow (`noexcept`) guarantee. For more information, see How to: Design for exception safety.
- Throw exceptions by value, catch them by reference. Don't catch what you can't handle.
- Don't use exception specifications, which are deprecated in C++11. For more information, see the Exception specifications and `noexcept` section.
- Use standard library exception types when they apply. Derive custom exception types from the `exception` Class hierarchy.
- Don't allow exceptions to escape from destructors or memory-deallocation functions.

[cpp_038] C++ - Exception Handling & RAII

Modern C++ Best Practices for Exceptions and Error Handling: Exceptions and performance

Sumber: ht

Exceptions and performance

The exception mechanism has a minimal performance cost if no exception is thrown. If an exception is thrown, the cost of the stack traversal and unwinding is roughly comparable to the cost of a function call. Other data structures are required to track the call stack after a **`try`** block is entered, and more instructions are required to unwind the stack if

an exception is thrown. However, in most scenarios, the cost in performance and memory footprint isn't significant. The adverse effect of exceptions on performance is likely to be significant only on memory-constrained systems. Or, in performance-critical loops, where an error is likely to occur regularly and there's tight coupling between the code to handle it and the code that reports it. In any case, it's impossible to know the actual cost of exceptions without profiling and measuring. Even in those rare cases when the cost is significant, you can weigh it against the increased correctness, easier maintainability, and other advantages that are provided by a well-designed exception policy.

[cpp_039] C++ - Exception Handling & RAII

Modern C++ Best Practices for Exceptions and Error Handling: Exceptions versus assertions

Sumber: ht

Exceptions versus assertions

Exceptions and asserts are two distinct mechanisms for detecting run-time errors in a program. Use `assert` statements to test for conditions during development that should always be true or always be false if all your code is correct. There's no point in handling such an error by using an exception, because the error indicates that something in the code has to be fixed. It doesn't represent a condition that the program has to recover from at run time. An `assert` stops execution at the statement so that you can inspect the program state in the debugger. An exception continues execution from the first appropriate catch handler. Use exceptions to check error conditions that might occur at run time even if your code is correct, for example, "file not found" or "out of memory." Exceptions can handle these conditions, even if the recovery just outputs a message to a log and ends the program. Always check arguments to public functions by using exceptions. Even if your function is error-free, you might not have complete control over arguments that a user might pass to it.

[cpp_040] C++ - Exception Handling & RAII

Modern C++ Best Practices for Exceptions and Error Handling: C++ exceptions versus Windows SEH exceptions

Sumber: ht

C++ exceptions versus Windows SEH exceptions

Both C and C++ programs can use the structured exception handling (SEH) mechanism in the Windows operating system. The concepts in SEH resemble the ones in C++ exceptions, except that SEH uses the **`__try`**, **`__except`**, and **`__finally`** constructs instead of **`try`** and **`catch`**. In the Microsoft C++ compiler (MSVC), C++ exceptions are implemented for SEH. However, when you write C++ code, use the C++ exception syntax.

For more information about SEH, see Structured Exception Handling (C/C++).

[cpp_041] C++ - Exception Handling & RAII

Modern C++ Best Practices for Exceptions and Error Handling: Exception specifications and `noexcept`

Sumber: ht

Exception specifications and `noexcept`

Exception specifications were introduced in C++ as a way to specify the exceptions that a function might throw. However, exception specifications proved problematic in practice, and are deprecated in the C++11 draft standard. We recommend that you don't use **`throw`** exception specifications except for `throw()`, which indicates that the function allows no exceptions to escape. If you must use exception specifications of the deprecated form `throw( type-name )`, MSVC support is limited. For more information, see Exception Specifications (throw). The **`noexcept`** specifier is introduced in C++11 as the preferred alternative to `throw()`.

[cpp_042] C++ - Exception Handling & RAII

Modern C++ Best Practices for Exceptions and Error Handling: See also

Sumber: ht

See also
How to: Interface between exceptional and non-exceptional code\
C++ language reference\
C++ Standard Library

[cpp_043] C++ - Modern Features - C++20 Modules

C++20 Modules (export module, import, Module partitions, Global module): Overview

Sumber: ht

`module`, `import`, `export`
The **`module`**, **`import`**, and **`export`** declarations are available in C++20 and require the compiler switch `/std:c++20` or later. For more information, see Overview of modules in C++.

[cpp_044] C++ - Modern Features - C++20 Modules

C++20 Modules (export module, import, Module partitions, Global module): `module`

Sumber: ht

`module`
Place a **`module`** declaration at the beginning of a module implementation file to specify that the file contents belong to the named module.

Contoh Kode / Perintah CLI:
```
module ModuleA;
```

[cpp_045] C++ - Modern Features - C++20 Modules

C++20 Modules (export module, import, Module partitions, Global module): `export`

Sumber: ht

`export`
Use an **`export module`** declaration for the module's primary interface file, which has an extension *`.ixx`* by default. If you want to use a different extension, use the /interface switch to compile it as a module interface.
In an interface file, use the **`export`** modifier on names that are intended to be part of the public interface:
Nonexported names aren't visible to code that imports the module:
The **`export`** keyword may not appear in a module implementation file. When **`export`** is applied to a namespace name, all names in the namespace are exported.

Contoh Kode / Perintah CLI:
```
export module ModuleA;

// ----------------------------------------

// ModuleA.ixx

export module ModuleA;

namespace ModuleA_NS
{
   export int f();
   export double d();
   double internal_f(); // not exported
}

// ----------------------------------------
```

```
import ModuleA;

int main() {
  ModuleA_NS::f(); // OK
  ModuleA_NS::d(); // OK
  ModuleA_NS::internal_f(); // Ill-formed: error C2065: 'internal_f': undeclared identifier
}
```

[cpp_046] C++ - Modern Features - C++20 Modules

C++20 Modules (export module, import, Module partitions, Global module): `import`

Sumber: ht

`import`

Use an **`import`** declaration to make a module's names visible in your program. The `import` declaration must appear after the `module` declaration and after any `#include` directives, but before any declarations in the file.

Contoh Kode / Perintah CLI:

```
module ModuleA;

#include "custom-lib.h"
import std;
import myModule;

// begin declarations here:
template <class T>
class Baz
{...};
```

[cpp_047] C++ - Modern Features - C++20 Modules

C++20 Modules (export module, import, Module partitions, Global module): Remarks

Sumber: ht

Remarks

Both **`import`** and **`module`** are treated as keywords only when they appear at the start of a logical line:

Microsoft Specific

In Microsoft C++, the tokens **`import`** and **`module`** are always identifiers and never keywords when they're used as arguments to a macro.

Contoh Kode / Perintah CLI:

```
// OK:
module ;
module module-name
import :
import <
import "
import module-name
export module ;
export module module-name
export import :
export import <
export import "
export import module-name

// Error:
int i; module ;
```

[cpp_048] C++ - Modern Features - C++20 Modules

C++20 Modules (export module, import, Module partitions, Global module): Example

Sumber: ht

Example
End Microsoft Specific

Contoh Kode / Perintah CLI:

```
#define foo(...) __VA_ARGS__
foo(
import // Always an identifier, never a keyword
)
```

[cpp_049] C++ - Modern Features - C++20 Modules

C++20 Modules (export module, import, Module partitions, Global module): See also

Sumber: ht

See also
Overview of modules in C++\
Import the C++ standard library using modules

[cpp_050] C++ - Lambda Expressions

Lambda Expressions (Capture clauses, Parameter lists, Mutable, Generic lambdas): Overview

Sumber: ht

Lambda expressions in C++
In C++11 and later, a lambda expression -- often called a *lambda* -- is a convenient way of defining an anonymous function object (a *closure*) right at the location where it's invoked or passed as an argument to a function. Typically lambdas are used to encapsulate a few lines of code that are passed to algorithms or asynchronous functions. This article defines what lambdas are, and compares them to other programming techniques. It describes their advantages, and provides some basic examples.

[cpp_051] C++ - Lambda Expressions

Lambda Expressions (Capture clauses, Parameter lists, Mutable, Generic lambdas): Related articles

Sumber: ht

Related articles
- Lambda expressions vs. function objects
- Working with lambda expressions
- constexpr lambda expressions

[cpp_052] C++ - Lambda Expressions

Lambda Expressions (Capture clauses, Parameter lists, Mutable, Generic lambdas): Parts of a lambda expression

Sumber: ht

Parts of a lambda expression
Here is a simple lambda that is passed as the third argument to the `std::sort()` function:
This illustration shows the parts of lambda syntax:
:::image type="complex" source="../cpp/media/lambdaexpsyntax.png" alt-text="Diagram that identifies the various parts of a lambda expression.":::
The lambda expression example is [=]() mutable throw() -> int { return x+y; } The [=] is the capture clause; also known as the lambda-introducer in the C++ specification. The parenthesis are for the parameter list. The mutable keyword is

optional. throw() is the optional exception specification. -> int is the optional trailing return type. The lambda body consists of the statement inside the curly braces, or return x+y; These are explained in more detail following the image.
:::image-end:::
1. *capture clause* (Also known as the *lambda-introducer* in the C++ specification.)
1. *parameter list* Optional. (Also known as the *lambda declarator*)
1. *mutable specification* Optional.
1. *exception-specification* Optional.
1. *trailing-return-type* Optional.
1. *lambda body*.

Contoh Kode / Perintah CLI:

```
#include <algorithm>
#include <cmath>

void abssort(float* x, unsigned n) {
    std::sort(x, x + n,
        // Lambda expression begins
        [](float a, float b) {
            return (std::abs(a) < std::abs(b));
        } // end of lambda expression
    );
}
```

[cpp_053] C++ - Lambda Expressions

Lambda Expressions (Capture clauses, Parameter lists, Mutable, Generic lambdas): Capture clause

Sumber: ht

Capture clause
A lambda can introduce new variables in its body (in **C++14**), and it can also access, or *capture*, variables from the surrounding scope. A lambda begins with the capture clause. It specifies which variables are captured, and whether the capture is by value or by reference. Variables that have the ampersand (`&`) prefix are accessed by reference and variables that don't have it are accessed by value.
An empty capture clause, `[ ]`, indicates that the body of the lambda expression accesses no variables in the enclosing scope.
You can use a capture-default mode to indicate how to capture any outside variables referenced in the lambda body: `[&]` means all variables that you refer to are captured by reference, and `[=]` means they're captured by value. You can use a default capture mode, and then specify the opposite mode explicitly for specific variables. For example, if a lambda body accesses the external variable `total` by reference and the external variable `factor` by value, then the following capture clauses are equivalent:
Only variables that are mentioned in the lambda body are captured when a capture-default is used.
If a capture clause includes a capture-default `&`, then no identifier in a capture of that capture clause can have the form `&identifier`. Likewise, if the capture clause includes a capture-default `=`, then no capture of that capture clause can have the form `=identifier`. An identifier or **`this`** can't appear more than once in a capture clause. The following code snippet illustrates some examples:
A capture followed by an ellipsis is a pack expansion, as shown in this variadic template example:
To use lambda expressions in the body of a class member function, pass the **`this`** pointer to the capture clause to provide acc

Contoh Kode / Perintah CLI:

```
[&total, factor]
[factor, &total]
[&, factor]
[=, &total]

// ----------------------------------------
```

```
struct S { void f(int i); };

void S::f(int i) {
    [&, i]{};      // OK
    [&, &i]{};     // ERROR: i preceded by & when & is the default
    [=, this]{};   // ERROR: this when = is the default
    [=, *this]{ }; // OK: captures this by value. See below.
    [i, i]{};      // ERROR: i repeated
}

// ----------------------------------------

template<class... Args>
void f(Args... args) {
    auto x = [args...] { return g(args...); };
    x();
}
```

[cpp_054] C++ - Lambda Expressions

Lambda Expressions (Capture clauses, Parameter lists, Mutable, Generic lambdas): Generalized capture (C++14)

Sumber: ht

Generalized capture (C++14)

In C++14, you can introduce and initialize new variables in the capture clause, without the need to have those variables exist in the lambda function's enclosing scope. The initialization can be expressed as any arbitrary expression; the type of the new variable is deduced from the type produced by the expression. This feature lets you capture move-only variables (such as `std::unique_ptr`) from the surrounding scope and use them in a lambda.

Contoh Kode / Perintah CLI:

```
pNums = make_unique<vector<int>>(nums);
//...
      auto a = [ptr = move(pNums)]()
        {
           // use ptr
        };
```

[cpp_055] C++ - Lambda Expressions

Lambda Expressions (Capture clauses, Parameter lists, Mutable, Generic lambdas): Parameter list

Sumber: ht

Parameter list

Lambdas can both capture variables and accept input parameters. A parameter list (*lambda declarator* in the Standard syntax) is optional and in most aspects resembles the parameter list for a function.

In **C++14**, if the parameter type is generic, you can use the **`auto`** keyword as the type specifier. This keyword tells the compiler to create the function call operator as a template. Each instance of **`auto`** in a parameter list is equivalent to a distinct type parameter.

A lambda expression can take another lambda expression as its argument. For more information, see "Higher-Order Lambda Expressions" in the article Examples of lambda expressions.

Because a parameter list is optional, you can omit the empty parentheses if you don't pass arguments to the lambda expression and its lambda-declarator doesn't contain *exception-specification*, *trailing-return-type*, or **`mutable`**.

Contoh Kode / Perintah CLI:

```
auto y = [] (int first, int second)
{
    return first + second;
```

```
};

// ----------------------------------------

auto y = [] (auto first, auto second)
{
    return first + second;
};
```

[cpp_056] C++ - Lambda Expressions

Lambda Expressions (Capture clauses, Parameter lists, Mutable, Generic lambdas): Mutable specification

Sumber: ht

Mutable specification

Typically, a lambda's function call operator is const-by-value, but use of the **`mutable`** keyword cancels this out. It doesn't produce mutable data members. The **`mutable`** specification enables the body of a lambda expression to modify variables that are captured by value. Some of the examples later in this article show how to use **`mutable`**.

[cpp_057] C++ - Lambda Expressions

Lambda Expressions (Capture clauses, Parameter lists, Mutable, Generic lambdas): Exception specification

Sumber: ht

Exception specification

You can use the **`noexcept`** exception specification to indicate that the lambda expression doesn't throw any exceptions. As with ordinary functions, the Microsoft C++ compiler generates warning C4297 if a lambda expression declares the **`noexcept`** exception specification and the lambda body throws an exception, as shown here:
For more information, see Exception specifications (throw).

Contoh Kode / Perintah CLI:

```
// throw_lambda_expression.cpp
// compile with: /W4 /EHsc
int main() // C4297 expected
{
   []() noexcept { throw 5; }();
}
```

[cpp_058] C++ - Lambda Expressions

Lambda Expressions (Capture clauses, Parameter lists, Mutable, Generic lambdas): Return type

Sumber: ht

Return type

The return type of a lambda expression is automatically deduced. You don't have to use the `auto` keyword unless you specify a *trailing-return-type*. The *trailing-return-type* resembles the return-type part of an ordinary function or member function. However, the return type must follow the parameter list, and you must include the trailing-return-type keyword **`->`** before the return type.
You can omit the return-type part of a lambda expression if the lambda body contains just one return statement. Or, if the expression doesn't return a value. If the lambda body contains one return statement, the compiler deduces the return type from the type of the return expression. Otherwise, the compiler deduces the return type as **`void`**. Consider the following example code snippets that illustrate this principle:
A lambda expression can produce another lambda expression as its return value. For more information, see "Higher-order lambda expressions" in Examples of lambda expressions.

Contoh Kode / Perintah CLI:

```
auto x1 = [](int i){ return i; }; // OK: return type is int
auto x2 = []{ return{ 1, 2 }; };  // ERROR: return type is void, deducing
                                  // return type from braced-init-list isn't valid
```

[cpp_059] C++ - Lambda Expressions

Lambda Expressions (Capture clauses, Parameter lists, Mutable, Generic lambdas): Lambda body

Sumber: ht

Lambda body

The lambda body of a lambda expression is a compound statement. It can contain anything that's allowed in the body of an ordinary function or member function. The body of both an ordinary function and a lambda expression can access these kinds of variables:

- Captured variables from the enclosing scope, as described previously.
- Parameters.
- Locally declared variables.
- Class data members, when declared inside a class and **`this`** is captured.
- Any variable that has static storage duration -- for example, global variables.

The following example contains a lambda expression that explicitly captures the variable `n` by value and implicitly captures the variable `m` by reference:

Because the variable `n` is captured by value, its value remains `0` after the call to the lambda expression. The **`mutable`** specification allows `n` to be modified within the lambda.

A lambda expression can only capture variables that have automatic storage duration. However, you can use variables that have static storage duration in the body of a lambda expression. The following example uses the `generate` function and a lambda expression to assign a value to each element in a `vector` object. The lambda expression modifies the static variable to generate the value of the next element.

For more information, see generate.

The following code example uses the function from the previous example, and adds an example of a lambda expression that uses the C++ Standard Library algorithm `generate_n`. This lambda expression assigns an element of a `vector` object to the sum of the previous two elements. The **`mutable`** keyword is used so that the body of the lambda expression can modify its copies of the external variables `x` and `y`, which the lambda expression captures by value.

Contoh Kode / Perintah CLI:

```
// captures_lambda_expression.cpp
// compile with: /W4 /EHsc
#include <iostream>
using namespace std;

int main()
{
   int m = 0;
   int n = 0;
   [&, n] (int a) mutable { m = ++n + a; }(4);
   cout << m << endl << n << endl;
}

// ----------------------------------------

Because the variable `n` is captured by value, its value remains `0` after the call to the lambda expression. The **`mutable`** specification allows `n` to be modified within the lambda.

A lambda expression can only capture variables that have automatic storage duration. However, you can use variables that have static storage duration in the body of a lambda expression. The following example uses the `generate` function and a lambda expression to assign a value to each element in a `vector` object. The lambda expression modifies the static variable to generate the value of the next element.

// ----------------------------------------
```

```
For more information, see [generate](../standard-library/algorithm-functions.md#generate).

The following code example uses the function from the previous example, and adds an example of a lambda expression that uses the C++ Standard Library algorithm `generate_n`. This lambda expression assigns an element of a `vector` object to the sum of the previous two elements. The **`mutable`** keyword is used so that the body of the lambda expression can modify its copies of the external variables `x` and `y`, which the lambda expression captures by value. Because the lambda expression captures the original variables `x` and `y` by value, their values remain `1` after the lambda executes.
```

[cpp_060] C++ - Lambda Expressions

Lambda Expressions (Capture clauses, Parameter lists, Mutable, Generic lambdas): `constexpr` lambda expressions

Sumber: ht

`constexpr` lambda expressions

Visual Studio 2017 version 15.3 and later (available in `/std:c++17` mode and later): You may declare a lambda expression as **`constexpr`** (or use it in a constant expression) when the initialization of each captured or introduced data member is allowed within a constant expression.

A lambda is implicitly **`constexpr`** if its result satisfies the requirements of a **`constexpr`** function:

If a lambda is implicitly or explicitly **`constexpr`**, conversion to a function pointer produces a **`constexpr`** function:

Contoh Kode / Perintah CLI:

```
int y = 32;
    auto answer = [y]() constexpr
    {
        int x = 10;
        return y + x;
    };

    constexpr int Increment(int n)
    {
        return [n] { return n + 1; }();
    }

// ----------------------------------------

auto answer = [](int n)
    {
        return 32 + n;
    };

    constexpr int response = answer(10);

// ----------------------------------------

auto Increment = [](int n)
    {
        return n + 1;
    };

    constexpr int(*inc)(int) = Increment;
```

[cpp_061] C++ - Lambda Expressions

Lambda Expressions (Capture clauses, Parameter lists, Mutable, Generic lambdas): Microsoft-specific

Sumber: ht

Microsoft-specific
Lambdas aren't supported in the following common language runtime (CLR) managed entities: **`ref class`**, **`ref struct`**, **`value class`**, or **`value struct`**.
If you're using a Microsoft-specific modifier such as `__declspec`, you can insert it into a lambda expression immediately after the `parameter-declaration-clause`. For example:
To determine whether a particular modifier is supported by lambdas, see the article about the modifier in the Microsoft-specific modifiers section.
Visual Studio supports C++11 Standard lambda functionality, and *stateless lambdas*. A stateless lambda is convertible to a function pointer that uses an arbitrary calling convention.

Contoh Kode / Perintah CLI:
```
auto Sqr = [](int t) __declspec(code_seg("PagedMem")) -> int { return t*t; };
```

[cpp_062] C++ - Lambda Expressions

Lambda Expressions (Capture clauses, Parameter lists, Mutable, Generic lambdas): See also

Sumber: ht

See also
C++ Language Reference
Function Objects in the C++ Standard Library
Function Call
`for_each`

[cpp_063] C++ - Move Semantics & Rvalue References

Move Constructors and Move Assignment Operators (std::move, Rvalue references &&): Overview

Sumber: ht

Move Constructors and Move Assignment Operators (C++)
This topic describes how to write a *move constructor* and a move assignment operator for a C++ class. A move constructor enables the resources owned by an rvalue object to be moved into an lvalue without copying. For more information about move semantics, see Rvalue Reference Declarator: &&.
This topic builds upon the following C++ class, `MemoryBlock`, which manages a memory buffer.
The following procedures describe how to write a move constructor and a move assignment operator for the example C++ class.

Contoh Kode / Perintah CLI:
```
// MemoryBlock.h
#pragma once
#include <iostream>
#include <algorithm>

class MemoryBlock
{
public:

   // Simple constructor that initializes the resource.
   explicit MemoryBlock(size_t length)
      : _length(length)
      , _data(new int[length])
```

```
   {
      std::cout << "In MemoryBlock(size_t). length = "
                << _length << "." << std::endl;
   }

   // Destructor.
   ~MemoryBlock()
   {
      std::cout << "In ~MemoryBlock(). length = "
                << _length << ".";

      if (_data != nullptr)
      {
         std::cout << " Deleting resource.";
         // Delete the resource.
         delete[] _data;
      }

      std::cout << std::endl;
   }

   // Copy constructor.
   MemoryBlock(const MemoryBlock& other)
      : _length(other._length)
      , _data(new int[other._length])
   {
      std::cout << "In MemoryBlock(const MemoryBlock&). length = "
                << other._length << ". Copying resource." << std::endl;

      std::copy(other._data, other._data + _length, _data);
   }

   // Copy assignment operator.
   MemoryBlock& operator=(const MemoryBlock& other)
   {
      std::cout << "In operator=(const MemoryBlock&). length = "
                << other._length << ". Copying resource." << std::endl;

      if (this != &other)
      {
         // Free the existing resource.
         delete[] _data;

         _length = other._length;
         _data = new int[_length];
         std::copy(other._data, other._data + _length, _data);
      }
      return *this;
   }

   // Retrieves the length of the data resource.
   size_t Length() const
   {
      return _length;
   }

private:
   size_t _length; // The length of the resource.
   int* _data; // The resource.
};
```

[cpp_064] C++ - Move Semantics & Rvalue References

Move Constructors and Move Assignment Operators (std::move, Rvalue references &&): To create a move constructor for a C++ class

Sumber: ht

To create a move constructor for a C++ class

1. Define an empty constructor method that takes an rvalue reference to the class type as its parameter, as demonstrated in the following example:

1. In the move constructor, assign the class data members from the source object to the object that is being constructed:

1. Assign the data members of the source object to default values. This prevents the destructor from freeing resources (such as memory) multiple times:

Contoh Kode / Perintah CLI:

```
MemoryBlock(MemoryBlock&& other)
       : _data(nullptr)
       , _length(0)
    {
    }

// -----------------------------------------

_data = other._data;
    _length = other._length;

// -----------------------------------------

other._data = nullptr;
    other._length = 0;
```

[cpp_065] C++ - Move Semantics & Rvalue References

Move Constructors and Move Assignment Operators (std::move, Rvalue references &&): To create a move assignment operator for a C++ class

Sumber: ht

To create a move assignment operator for a C++ class

1. Define an empty assignment operator that takes an rvalue reference to the class type as its parameter and returns a reference to the class type, as demonstrated in the following example:

1. In the move assignment operator, add a conditional statement that performs no operation if you try to assign the object to itself.

1. In the conditional statement, free any resources (such as memory) from the object that is being assigned to.

The following example frees the `_data` member from the object that is being assigned to:

Follow steps 2 and 3 in the first procedure to transfer the data members from the source object to the object that is being constructed:

1. Return a reference to the current object, as shown in the following example:

Contoh Kode / Perintah CLI:

```
MemoryBlock& operator=(MemoryBlock&& other)
    {
    }

// -----------------------------------------

if (this != &other)
    {
    }

// -----------------------------------------
```

```
// Free the existing resource.
    delete[] _data;
```

[cpp_066] C++ - Move Semantics & Rvalue References

Move Constructors and Move Assignment Operators (std::move, Rvalue references &&): Example: Complete move constructor and assignment operator

Sumber: ht

Example: Complete move constructor and assignment operator

The following example shows the complete move constructor and move assignment operator for the `MemoryBlock` class:

Contoh Kode / Perintah CLI:

```
// Move constructor.
MemoryBlock(MemoryBlock&& other) noexcept
   : _data(nullptr)
   , _length(0)
{
   std::cout << "In MemoryBlock(MemoryBlock&&). length = "
             << other._length << ". Moving resource." << std::endl;

   // Copy the data pointer and its length from the
   // source object.
   _data = other._data;
   _length = other._length;

   // Release the data pointer from the source object so that
   // the destructor does not free the memory multiple times.
   other._data = nullptr;
   other._length = 0;
}

// Move assignment operator.
MemoryBlock& operator=(MemoryBlock&& other) noexcept
{
   std::cout << "In operator=(MemoryBlock&&). length = "
             << other._length << "." << std::endl;

   if (this != &other)
   {
      // Free the existing resource.
      delete[] _data;

      // Copy the data pointer and its length from the
      // source object.
      _data = other._data;
      _length = other._length;

      // Release the data pointer from the source object so that
      // the destructor does not free the memory multiple times.
      other._data = nullptr;
      other._length = 0;
   }
   return *this;
}
```

[cpp_067] C++ - Move Semantics & Rvalue References

Move Constructors and Move Assignment Operators (std::move, Rvalue references &&): Example Use move semantics to improve performance

Sumber: ht

Example Use move semantics to improve performance

The following example shows how move semantics can improve the performance of your applications. The example adds two elements to a vector object and then inserts a new element between the two existing elements. The `vector` class uses move semantics to perform the insertion operation efficiently by moving the elements of the vector instead of copying them.

This example produces the following output:

Before Visual Studio 2010, this example produced the following output:

The version of this example that uses move semantics is more efficient than the version that does not use move semantics because it performs fewer copy, memory allocation, and memory deallocation operations.

Contoh Kode / Perintah CLI:

```
// rvalue-references-move-semantics.cpp
// compile with: /EHsc
#include "MemoryBlock.h"
#include <vector>

using namespace std;

int main()
{
   // Create a vector object and add a few elements to it.
   vector<MemoryBlock> v;
   v.push_back(MemoryBlock(25));
   v.push_back(MemoryBlock(75));

   // Insert a new element into the second position of the vector.
   v.insert(v.begin() + 1, MemoryBlock(50));
}

// ----------------------------------------

Before Visual Studio 2010, this example produced the following output:
```

[cpp_068] C++ - Move Semantics & Rvalue References

Move Constructors and Move Assignment Operators (std::move, Rvalue references &&): Robust Programming

Sumber: ht

Robust Programming

To prevent resource leaks, always free resources (such as memory, file handles, and sockets) in the move assignment operator.

To prevent the unrecoverable destruction of resources, properly handle self-assignment in the move assignment operator.

If you provide both a move constructor and a move assignment operator for your class, you can eliminate redundant code by writing the move constructor to call the move assignment operator. The following example shows a revised version of the move constructor that calls the move assignment operator:

The std::move function converts the lvalue `other` to an rvalue.

Contoh Kode / Perintah CLI:

```
// Move constructor.
MemoryBlock(MemoryBlock&& other) noexcept
   : _data(nullptr)
   , _length(0)
```

```
{
   *this = std::move(other);
}
```

[cpp_069] C++ - Move Semantics & Rvalue References

Move Constructors and Move Assignment Operators (std::move, Rvalue references &&): See also

Sumber: ht

See also
Rvalue Reference Declarator: &&
std::move

[cpp_070] C++ - Templates & Generic Programming

Class Templates & Generic Programming (template parameters, Specialization): Overview

Sumber: ht

Class Templates
This article describes rules that are specific to C++ class templates.

[cpp_071] C++ - Templates & Generic Programming

Class Templates & Generic Programming (template parameters, Specialization): Member functions of class templates

Sumber: ht

Member functions of class templates
Member functions can be defined inside or outside of a class template. They're defined like function templates if defined outside the class template.
As with any template class member function, the definition of the class's constructor member function includes the template argument list twice.
Member functions can themselves be function templates and specify extra parameters, as in the following example.

Contoh Kode / Perintah CLI:

```
// member_function_templates1.cpp
template<class T, int i> class MyStack
{
    T*  pStack;
    T StackBuffer[i];
    static const int cItems = i * sizeof(T);
public:
    MyStack( void );
    void push( const T item );
    T& pop( void );
};

template< class T, int i > MyStack< T, i >::MyStack( void )
{
}

template< class T, int i > void MyStack< T, i >::push( const T item )
{
}

template< class T, int i > T& MyStack< T, i >::pop( void )
{
}
```

```
int main()
{
}

// ----------------------------------------

// member_templates.cpp
template<typename T>
class X
{
public:
   template<typename U>
   void mf(const U &u);
};

template<typename T> template <typename U>
void X<T>::mf(const U &u)
{
}

int main()
{
}
```

[cpp_072] C++ - Templates & Generic Programming

Class Templates & Generic Programming (template parameters, Specialization): Nested class templates

Sumber: ht

Nested class templates

Templates can be defined within classes or class templates, in which case they're referred to as member templates. Member templates that are classes are referred to as nested class templates. Member templates that are functions are discussed in Member Function Templates.

Nested class templates are declared as class templates inside the scope of the outer class. They can be defined inside or outside of the enclosing class.

The following code demonstrates a nested class template inside an ordinary class.

The following code uses nested template type parameters to create nested class templates:

Local classes aren't allowed to have member templates.

Contoh Kode / Perintah CLI:

```
// nested_class_template1.cpp
// compile with: /EHsc
#include <iostream>
using namespace std;

class X
{
   template <class T>
   struct Y
   {
      T m_t;
      Y(T t): m_t(t) { }
   };

   Y<int> yInt;
   Y<char> yChar;

public:
```

```
   X(int i, char c) : yInt(i), yChar(c) { }
   void print()
   {
      cout << yInt.m_t << " " << yChar.m_t << endl;
   }
};

int main()
{
   X x(1, 'a');
   x.print();
}

// ----------------------------------------

// nested_class_template2.cpp
// compile with: /EHsc
#include <iostream>
using namespace std;

template <class T>
class X
{
   template <class U> class Y
   {
      U* u;
   public:
      Y();
      U& Value();
      void print();
      ~Y();
   };

   Y<int> y;
public:
   X(T t) { y.Value() = t; }
   void print() { y.print(); }
};

template <class T>
template <class U>
X<T>::Y<U>::Y()
{
   cout << "X<T>::Y<U>::Y()" << endl;
   u = new U();
}

template <class T>
template <class U>
U& X<T>::Y<U>::Value()
{
   return *u;
}

template <class T>
template <class U>
void X<T>::Y<U>::print()
{
   cout << this->Value() << endl;
}

template <class T>
```

```
template <class U>
X<T>::Y<U>::~Y()
{
   cout << "X<T>::Y<U>::~Y()" << endl;
   delete u;
}

int main()
{
   X<int>* xi = new X<int>(10);
   X<char>* xc = new X<char>('c');
   xi->print();
   xc->print();
   delete xi;
   delete xc;
}

/* Output:
X<T>::Y<U>::Y()
X<T>::Y<U>::Y()
10
99
X<T>::Y<U>::~Y()
X<T>::Y<U>::~Y()
*/
```

[cpp_073] C++ - Templates & Generic Programming

Class Templates & Generic Programming (template parameters, Specialization): Template friends

Sumber: ht

Template friends

Class templates can have friends. A class or class template, function, or function template can be a friend to a template class. Friends can also be specializations of a class template or function template, but not partial specializations.

In the following example, a friend function is defined as a function template within the class template. This code produces a version of the friend function for every instantiation of the template. This construct is useful if your friend function depends on the same template parameters as the class does.

The next example involves a friend that has a template specialization. A function template specialization is automatically a friend if the original function template is a friend.

It's also possible to declare only the specialized version of the template as the friend, as the comment before the friend declaration in the following code indicates. If you declare a specialization as a friend, you must put the definition of the friend template specialization outside of the template class.

The next example shows a friend class template declared within a class template. The class template is then used as the template argument for the friend class. Friend class templates must be defined outside of the class template in which they're declared. Any specializations or partial specializations of the friend template are also friends of the original class template.

Contoh Kode / Perintah CLI:

```
// template_friend1.cpp
// compile with: /EHsc

#include <iostream>
using namespace std;

template <class T> class Array {
   T* array;
   int size;
```

```
public:
   Array(int sz): size(sz) {
      array = new T[size];
      memset(array, 0, size * sizeof(T));
   }

   Array(const Array& a) {
      size = a.size;
      array = new T[size];
      memcpy_s(array, a.array, sizeof(T));
   }

   T& operator[](int i) {
      return *(array + i);
   }

   int Length() { return size; }

   void print() {
      for (int i = 0; i < size; i++)
         cout << *(array + i) << " ";

      cout << endl;
   }

   template<class T>
   friend Array<T>* combine(Array<T>& a1, Array<T>& a2);
};

template<class T>
Array<T>* combine(Array<T>& a1, Array<T>& a2) {
   Array<T>* a = new Array<T>(a1.size + a2.size);
   for (int i = 0; i < a1.size; i++)
      (*a)[i] = *(a1.array + i);

   for (int i = 0; i < a2.size; i++)
      (*a)[i + a1.size] = *(a2.array + i);

   return a;
}

int main() {
   Array<char> alpha1(26);
   for (int i = 0 ; i < alpha1.Length() ; i++)
      alpha1[i] = 'A' + i;

   alpha1.print();

   Array<char> alpha2(26);
   for (int i = 0 ; i < alpha2.Length() ; i++)
      alpha2[i] = 'a' + i;

   alpha2.print();
   Array<char>*alpha3 = combine(alpha1, alpha2);
   alpha3->print();
   delete alpha3;
}
/* Output:
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
a b c d e f g h i j k l m n o p q r s t u v w x y z
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z a b c d e f g h i j k l m n o p q r s t u v w x y z
*/
```

```
// ----------------------------------------

// template_friend2.cpp
// compile with: /EHsc
#include <iostream>
using namespace std;

template <class T>
class Array;

template <class T>
void f(Array<T>& a);

template <class T> class Array
{
    T* array;
    int size;

public:
    Array(int sz): size(sz)
    {
        array = new T[size];
        memset(array, 0, size * sizeof(T));
    }
    Array(const Array& a)
    {
        size = a.size;
        array = new T[size];
        memcpy_s(array, a.array, sizeof(T));
    }
    T& operator[](int i)
    {
        return *(array + i);
    }
    int Length()
    {
        return size;
    }
    void print()
    {
        for (int i = 0; i < size; i++)
        {
            cout << *(array + i) << " ";
        }
        cout << endl;
    }
    // If you replace the friend declaration with the int-specific
    // version, only the int specialization will be a friend.
    // The code in the generic f will fail
    // with C2248: 'Array<T>::size' :
    // cannot access private member declared in class 'Array<T>'.
    //friend void f<int>(Array<int>& a);

    friend void f<>(Array<T>& a);
};

// f function template, friend of Array<T>
template <class T>
void f(Array<T>& a)
{
    cout << a.size << " generic" << endl;
```

```
}

// Specialization of f for int arrays
// will be a friend because the template f is a friend.
template<> void f(Array<int>& a)
{
    cout << a.size << " int" << endl;
}

int main()
{
    Array<char> ac(10);
    f(ac);

    Array<int> a(10);
    f(a);
}
/* Output:
10 generic
10 int
*/

// ----------------------------------------

// template_friend3.cpp
// compile with: /EHsc
#include <iostream>
using namespace std;

template <class T>
class X
{
private:
   T* data;
   void InitData(int seed) { data = new T(seed); }
public:
   void print() { cout << *data << endl; }
   template <class U> friend class Factory;
};

template <class U>
class Factory
{
public:
   U* GetNewObject(int seed)
   {
      U* pu = new U;
      pu->InitData(seed);
      return pu;
   }
};

int main()
{
   Factory< X<int> > XintFactory;
   X<int>* x1 = XintFactory.GetNewObject(65);
   X<int>* x2 = XintFactory.GetNewObject(97);

   Factory< X<char> > XcharFactory;
   X<char>* x3 = XcharFactory.GetNewObject(65);
   X<char>* x4 = XcharFactory.GetNewObject(97);
   x1->print();
```

```
   x2->print();
   x3->print();
   x4->print();
}
/* Output:
65
97
A
a
*/
```

[cpp_074] C++ - Templates & Generic Programming

Class Templates & Generic Programming (template parameters, Specialization): Reuse of Template Parameters

Sumber: ht

Reuse of Template Parameters

Template parameters can be reused in the template parameter list. For example, the following code is allowed:

Contoh Kode / Perintah CLI:

```
// template_specifications2.cpp

class Y
{
};
template<class T, T* pT> class X1
{
};
template<class T1, class T2 = T1> class X2
{
};

Y aY;

X1<Y, &aY> x1;
X2<int> x2;

int main()
{
}
```